

Объектно-ориентированное программирование

Лекция 12: Структурные паттерны (Часть 2)

Composite (Компоновщик). Decorator (Декоратор, в т.ч. функции-декораторы).
Flyweight (Приспособленец).

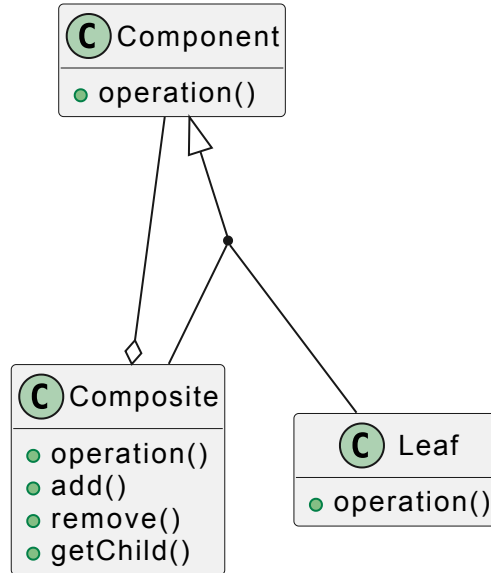


Компоновщик - паттерн, структурирующий объекты.

Компонует объекты в древовидные структуры для представления иерархий часть-целое. Позволяет клиентам единообразно трактовать индивидуальные и составные объекты.

Используйте паттерн компоновщик, когда:

- нужно представить иерархию объектов вида часть-целое;
- хотите, чтобы клиенты единообразно трактовали составные и индивидуальные объекты.



```
1  from abc import ABC, abstractmethod
2
3  class Component(ABC):
4      @abstractmethod
5      def get_size(self): pass
6
7  class File(Component):
8      def __init__(self, name, size):
9          self.name = name
10         self.size = size
11
12         def get_size(self):
13             return self.size
14
15  class Folder(Component):
```

```
1  from abc import ABC, abstractmethod
2
3  class Component(ABC):
4      @abstractmethod
5      def get_size(self): pass
6
7  class File(Component):
8      def __init__(self, name, size):
9          self.name = name
10         self.size = size
11
12     def get_size(self):
13         return self.size
14
15 class Folder(Component):
```

```
3 class Component(ABC):
4     @abstractmethod
5     def get_size(self): pass
6
7 class File(Component):
8     def __init__(self, name, size):
9         self.name = name
10        self.size = size
11
12    def get_size(self):
13        return self.size
14
15 class Folder(Component):
16    def __init__(self, name):
17        self.name = name
```

```
6
7 class File(Component):
8     def __init__(self, name, size):
9         self.name = name
10        self.size = size
11
12    def get_size(self):
13        return self.size
14
15 class Folder(Component):
16    def __init__(self, name):
17        self.name = name
18        self.children = []
19
20    def add(self, component: Component):
```



```
14
15 class Folder(Component):
16     def __init__(self, name):
17         self.name = name
18         self.children = []
19
20     def add(self, component: Component):
21         self.children.append(component)
22
23     def get_size(self):
24         total = sum(child.get_size() for child in self.children)
25         print(f"Папка {self.name}: {total} байт")
26         return total
27
28 # Использование
```



Паттерн Компоновщик (Composite)

```
8     def __init__(self, name, size):
9         self.name = name
10        self.size = size
11
12    def get_size(self):
13        return self.size
14
15 class Folder(Component):
16     def __init__(self, name):
17         self.name = name
18         self.children = []
19
20     def add(self, component: Component):
21         self.children.append(component)
22
```

```
13         return self.size
14
15 class Folder(Component):
16     def __init__(self, name):
17         self.name = name
18         self.children = []
19
20     def add(self, component: Component):
21         self.children.append(component)
22
23     def get_size(self):
24         total = sum(child.get_size() for child in self.children)
25         print(f"Папка {self.name}: {total} байт")
26         return total
27
```

```
18         self.children = []
19
20     def add(self, component: Component):
21         self.children.append(component)
22
23     def get_size(self):
24         total = sum(child.get_size() for child in self.children)
25         print(f"Папка {self.name}: {total} байт")
26         return total
27
28 # Использование
29 root = Folder("root")
30 img = File("photo.jpg", 1024)
31 doc = File("resume.pdf", 512)
32
```

```
17         self.name = name
18         self.children = []
19
20     def add(self, component: Component):
21         self.children.append(component)
22
23     def get_size(self):
24         total = sum(child.get_size() for child in self.children)
25         print(f"Папка {self.name}: {total} байт")
26         return total
27
28 # Использование
29 root = Folder("root")
30 img = File("photo.jpg", 1024)
31 doc = File("resume.pdf", 512)
```

```
25         print(f'Папка {sect.name}: {total} байт')
26     return total
27
28 # Использование
29 root = Folder("root")
30 img = File("photo.jpg", 1024)
31 doc = File("resume.pdf", 512)
32
33 subfolder = Folder("docs")
34 subfolder.add(doc)
35
36 root.add(img)
37 root.add(subfolder)
38
39 root.get_size()
```

Паттерн Компоновщик (Composite). Результаты

- определяет иерархии классов, состоящие из примитивных и составных объектов. Из примитивных объектов можно составлять более сложные, которые, в свою очередь, участвуют в более сложных композициях и так далее. Любой клиент, ожидающий примитивного объекта, может работать и с составным;
- упрощает архитектуру клиента. Клиенты могут единообразно работать с индивидуальными объектами и с составными структурами. Обычно клиенту неизвестно, взаимодействует ли он с листовым или составным объектом. Это упрощает код клиента, поскольку нет необходимости писать функции, ветвящиеся в зависимости от того, с объектом какого класса они работают;

Паттерн Компоновщик (Composite). Результаты

- определяет иерархии классов, состоящие из примитивных и составных объектов. Из примитивных объектов можно составлять более сложные, которые, в свою очередь, участвуют в более сложных композициях и так далее. Любой клиент, ожидающий примитивного объекта, может работать и с составным;
- упрощает архитектуру клиента. Клиенты могут единообразно работать с индивидуальными объектами и с составными структурами. Обычно клиенту неизвестно, взаимодействует ли он с листовым или составным объектом. Это упрощает код клиента, поскольку нет необходимости писать функции, ветвящиеся в зависимости от того, с объектом какого класса они работают;

Паттерн Компоновщик (Composite). Результаты

- определяет иерархии классов, состоящие из примитивных и составных объектов. Из примитивных объектов можно составлять более сложные, которые, в свою очередь, участвуют в более сложных композициях и так далее. Любой клиент, ожидающий примитивного объекта, может работать и с составным;
- упрощает архитектуру клиента. Клиенты могут единообразно работать с индивидуальными объектами и с составными структурами. Обычно клиенту неизвестно, взаимодействует ли он с листовым или составным объектом. Это упрощает код клиента, поскольку нет необходимости писать функции, ветвящиеся в зависимости от того, с объектом какого класса они работают;

Паттерн Компоновщик (Composite). Результаты

- облегчает добавление новых видов компонентов. Новые подклассы будут автоматически работать с уже существующими структурами и клиентским кодом. Изменять клиента при добавлении новых компонентов не нужно;
- способствует созданию общего дизайна. Однако такая простота добавления новых компонентов имеет и свои отрицательные стороны: становится трудно наложить ограничения на то, какие объекты могут входить в состав композиции. Иногда желательно, чтобы составной объект мог включать только определенные виды компонентов. Паттерн компоновщик не позволяет воспользоваться для реализации таких ограничений статической системой типов. Вместо этого следует проводить проверки во время выполнения.

- **облегчает добавление новых видов компонентов.** Новые подклассы будут автоматически работать с уже существующими структурами и клиентским кодом. Изменять клиента при добавлении новых компонентов не нужно;
- способствует созданию общего дизайна. Однако такая простота добавления новых компонентов имеет и свои отрицательные стороны: становится трудно наложить ограничения на то, какие объекты могут входить в состав композиции. Иногда желательно, чтобы составной объект мог включать только определенные виды компонентов. Паттерн компоновщик не позволяет воспользоваться для реализации таких ограничений статической системой типов. Вместо этого следует проводить проверки во время выполнения.

Паттерн Компоновщик (Composite). Результаты

- **облегчает добавление новых видов компонентов.** Новые подклассы будут автоматически работать с уже существующими структурами и клиентским кодом. Изменять клиента при добавлении новых компонентов не нужно;
- **способствует созданию общего дизайна.** Однако такая простота добавления новых компонентов имеет и свои отрицательные стороны: становится трудно наложить ограничения на то, какие объекты могут входить в состав композиции. Иногда желательно, чтобы составной объект мог включать только определенные виды компонентов. Паттерн компоновщик не позволяет воспользоваться для реализации таких ограничений статической системой типов. Вместо этого следует проводить проверки во время выполнения.

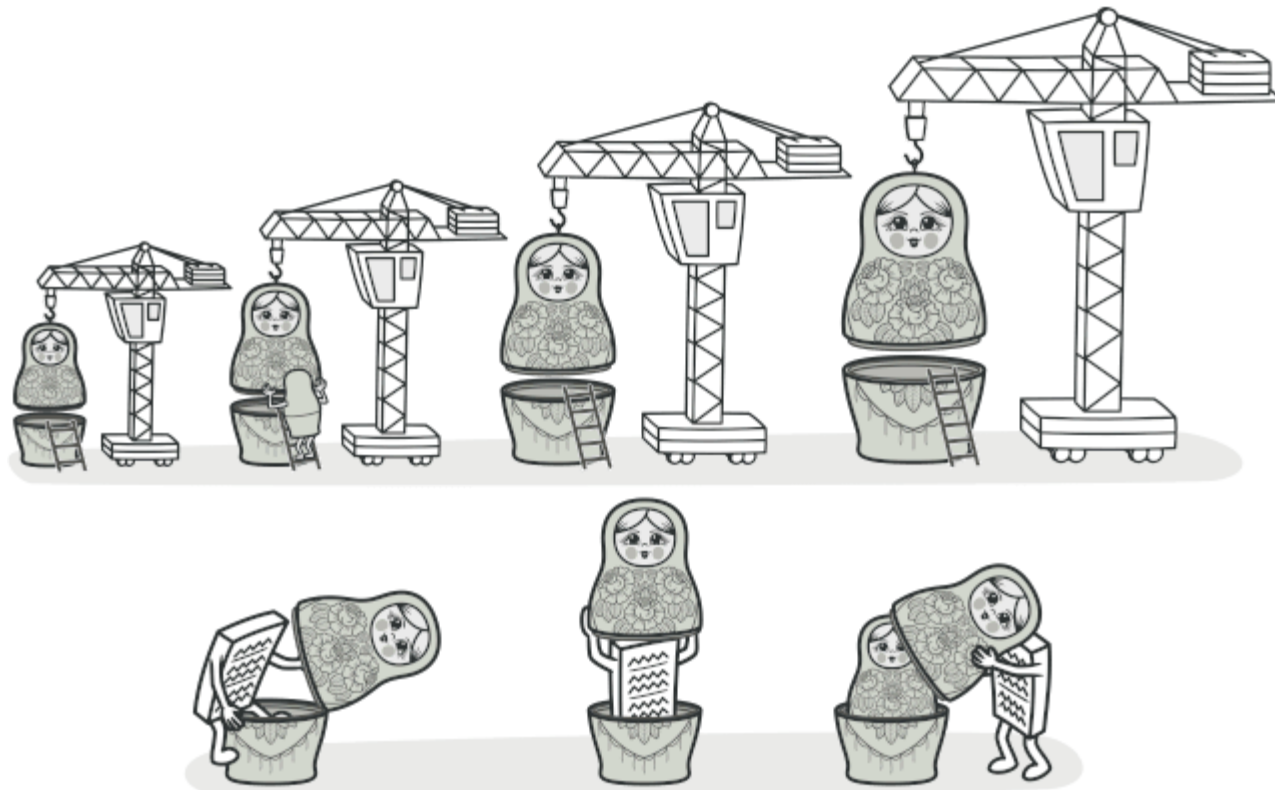
Паттерн Компоновщик (Composite). Результаты

- **облегчает добавление новых видов компонентов.** Новые подклассы будут автоматически работать с уже существующими структурами и клиентским кодом. Изменять клиента при добавлении новых компонентов не нужно;
- **способствует созданию общего дизайна.** Однако такая простота добавления новых компонентов имеет и свои отрицательные стороны: становится **трудно наложить ограничения на то, какие объекты могут входить в состав композиции.** Иногда желательно, чтобы составной объект мог включать только определенные виды компонентов. Паттерн компоновщик не позволяет воспользоваться для реализации таких ограничений статической системой типов. Вместо этого следует проводить проверки во время выполнения.



Университет
Сириус
Колледж

Паттерн Декоратор (Decorator)



Декоратор - паттерн, структурирующий объекты.

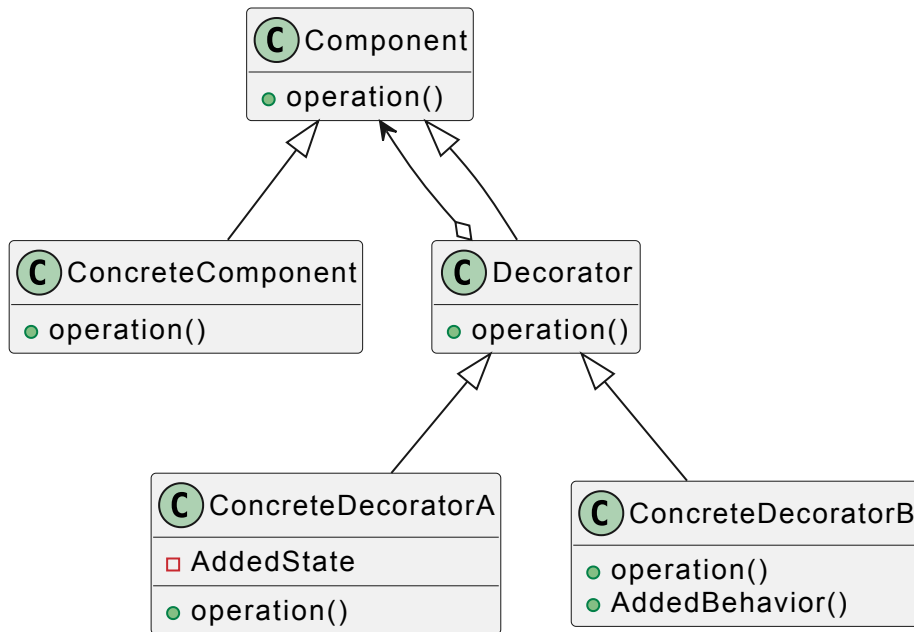
Динамически добавляет объекту новые обязанности. Является гибкой альтернативой порождению подклассов с целью расширения функциональности.

Используйте паттерн декоратор:

- для динамического, прозрачного для клиентов добавления обязанностей объектам;
- для реализации обязанностей, которые могут быть сняты с объекта;
- когда расширение путем порождения подклассов по каким-то причинам неудобно или невозможно. Иногда приходится реализовывать много независимых расширений, так что порождение подклассов для поддержки всех возможных комбинаций приведет к комбинаторному росту их числа. В других случаях определение класса может быть скрыто или почему-либо еще недоступно, так что породить от него подкласс нельзя.



Паттерн Декоратор (Decorator)





Паттерн Декоратор (Decorator)

```
1  from abc import ABC, abstractmethod
2  import base64
3  import zlib
4
5  # 1. Общий интерфейс
6  class DataSource(ABC):
7      @abstractmethod
8      def write(self, data: str): pass
9
10     @abstractmethod
11     def read(self) -> str: pass
12
13 # 2. Базовый компонент (Простая запись в файл)
14 class FileDataSource(DataSource):
15     def init (self filename):
```



Паттерн Декоратор (Decorator)

```
1  from abc import ABC, abstractmethod
2  import base64
3  import zlib
4
5  # 1. Общий интерфейс
6  class DataSource(ABC):
7      @abstractmethod
8      def write(self, data: str): pass
9
10     @abstractmethod
11     def read(self) -> str: pass
12
13 # 2. Базовый компонент (Простая запись в файл)
14 class FileDataSource(DataSource):
15     def init(self, filename):
```



Паттерн Декоратор (Decorator)

```
11     def read(self) -> str: pass
12
13 # 2. Базовый компонент (Простая запись в файл)
14 class FileDataSource(DataSource):
15     def __init__(self, filename):
16         self.filename = filename
17
18     def write(self, data: str):
19         print(f"[Disk] Запись в файл {self.filename}: {data}")
20
21     def read(self) -> str:
22         print(f"[Disk] Чтение из файла {self.filename}")
23         return "data"
24
25 # 3. Базовый Декоратор (принимает объект-источник)
```



Паттерн Декоратор (Decorator)

```
23         return "data"
24
25 # 3. Базовый Декоратор (принимает объект-источник)
26 class DataSourceDecorator(DataSource):
27     def __init__(self, source: DataSource):
28         self.wrappee = source
29
30     def write(self, data: str):
31         self.wrappee.write(data)
32
33     def read(self) -> str:
34         return self.wrappee.read()
35
36 # 4. Конкретный Декоратор: Шифрование
37 class EncryptionDecorator(DataSourceDecorator):
```



Паттерн Декоратор (Decorator)

```
21     def read(self) -> str:
22         print(f"[Disk] Чтение из файла {self.filename}")
23         return "data"
24
25 # 3. Базовый Декоратор (принимает объект-источник)
26 class DataSourceDecorator(DataSource):
27     def __init__(self, source: DataSource):
28         self.wrappee = source
29
30     def write(self, data: str):
31         self.wrappee.write(data)
32
33     def read(self) -> str:
34         return self.wrappee.read()
35
```



Паттерн Декоратор (Decorator)

```
27     def __init__(self, source: DataSource):
28         self.wrappee = source
29
30     def write(self, data: str):
31         self.wrappee.write(data)
32
33     def read(self) -> str:
34         return self.wrappee.read()
35
36 # 4. Конкретный Декоратор: Шифрование
37 class EncryptionDecorator(DataSourceDecorator):
38     def write(self, data: str):
39         encrypted = f"ENC({data[::-1]})"
40         print(f" -> Шифруем: {encrypted}")
41         super().write(encrypted)
```



Паттерн Декоратор (Decorator)

```
32
33     def read(self) -> str:
34         return self.wrappee.read()
35
36 # 4. Конкретный Декоратор: Шифрование
37 class EncryptionDecorator(DataSourceDecorator):
38     def write(self, data: str):
39         encrypted = f"ENC({data[::-1]})"
40         print(f" -> Шифруем: {encrypted}")
41         super().write(encrypted)
42
43 # 5. Конкретный Декоратор: Сжатие
44 class CompressionDecorator(DataSourceDecorator):
45     def write(self, data: str):
46         # Имитация сжатия
```



Паттерн Декоратор (Decorator)

```
33     def read(self) -> str:
34         return self.wrappee.read()
35
36 # 4. Конкретный Декоратор: Шифрование
37 class EncryptionDecorator(DataSourceDecorator):
38     def write(self, data: str):
39         encrypted = f"ENC({data[::-1]})"
40         print(f" -> Шифруем: {encrypted}")
41         super().write(encrypted)
42
43 # 5. Конкретный Декоратор: Сжатие
44 class CompressionDecorator(DataSourceDecorator):
45     def write(self, data: str):
46         # Имитация сжатия
47         compressed = f"ZIP({data[:5]}...)"
```




Паттерн Декоратор (Decorator)

```
34         return self.wrappee.read()
35
36 # 4. Конкретный Декоратор: Шифрование
37 class EncryptionDecorator(DataSourceDecorator):
38     def write(self, data: str):
39         encrypted = f"ENC({data[::-1]})"
40         print(f" -> Шифруем: {encrypted}")
41         super().write(encrypted)
42
43 # 5. Конкретный Декоратор: Сжатие
44 class CompressionDecorator(DataSourceDecorator):
45     def write(self, data: str):
46         # Имитация сжатия
47         compressed = f"ZIP({data[:5]}...)"
48         print(f" -> Сжимаем: {compressed}")
```



Паттерн Декоратор (Decorator)

```
39         encrypted = f"ENC({data[:: -1]})"
40         print(f" -> Шифруем: {encrypted}")
41         super().write(encrypted)
42
43 # 5. Конкретный Декоратор: Сжатие
44 class CompressionDecorator(DataSourceDecorator):
45     def write(self, data: str):
46         # Имитация сжатия
47         compressed = f"ZIP({data[:5]}...)"
48         print(f" -> Сжимаем: {compressed}")
49         super().write(compressed)
50
51 # --- Использование ---
52
53 # Клиент настраивает "слоеный пирог":
```



Паттерн Декоратор (Decorator)

```
47         compressed = zlib.compress(data[:5])
48         print(f" -> Сжимаем: {compressed}")
49         super().write(compressed)
50
51 # --- Использование ---
52
53 # Клиент настраивает "слоеный пирог":
54 # Данные -> Сжать -> Зашифровать -> Записать на диск
55 source = FileDataSource("secret_data.txt")
56 source = EncryptionDecorator(source)
57 source = CompressionDecorator(source)
58 # Обратите внимание: порядок обертки влияет на порядок выполнения!
59
60 print("--- Процесс записи ---")
61 source.write("MySensitiveInformation")
```



Паттерн Декоратор (Decorator)

```
47         compressed = zlib.compress(data[:5])
48         print(f" -> Сжимаем: {compressed}")
49         super().write(compressed)
50
51 # --- Использование ---
52
53 # Клиент настраивает "слоеный пирог":
54 # Данные -> Сжать -> Зашифровать -> Записать на диск
55 source = FileDataSource("secret_data.txt")
56 source = EncryptionDecorator(source)
57 source = CompressionDecorator(source)
58 # Обратите внимание: порядок обертки влияет на порядок выполнения!
59
60 print("--- Процесс записи ---")
61 source.write("MySensitiveInformation")
```



Паттерн Декоратор (Decorator)

```
47         compressed = zlib.compress(data[:5])
48         print(f" -> Сжимаем: {compressed}")
49         super().write(compressed)
50
51 # --- Использование ---
52
53 # Клиент настраивает "слоеный пирог":
54 # Данные -> Сжать -> Зашифровать -> Записать на диск
55 source = FileDataSource("secret_data.txt")
56 source = EncryptionDecorator(source)
57 source = CompressionDecorator(source)
58 # Обратите внимание: порядок обертки влияет на порядок выполнения!
59
60 print("--- Процесс записи ---")
61 source.write("MySensitiveInformation")
```

Паттерн Декоратор (Decorator). Результаты

- Большая гибкость, нежели у статического наследования. Паттерн декоратор позволяет более гибко добавлять объекту новые обязанности, чем было бы возможно в случае статического (множественного) наследования. Декоратор может добавлять и удалять обязанности во время выполнения программы. При использовании же наследования требуется создавать новый класс для каждой дополнительной обязанности, что ведет к увеличению числа классов и, как следствие, к возрастанию сложности системы. Кроме того, применение нескольких декораторов к одному компоненту позволяет произвольным образом сочетать обязанности. Декораторы позволяют легко добавить одно и то же свойство дважды, но это может быть чревато ошибками;

- Большая гибкость, нежели у статического наследования. Паттерн декоратор позволяет более гибко добавлять объекту новые обязанности, чем было бы возможно в случае статического (множественного) наследования. Декоратор может добавлять и удалять обязанности во время выполнения программы. При использовании же наследования требуется создавать новый класс для каждой дополнительной обязанности, что ведет к увеличению числа классов и, как следствие, к возрастанию сложности системы. Кроме того, применение нескольких декораторов к одному компоненту позволяет произвольным образом сочетать обязанности. Декораторы позволяют легко добавить одно и то же свойство дважды, но это может быть чревато ошибками;

Паттерн Декоратор (Decorator). Результаты

- позволяет избежать перегруженных функциями классов на верхних уровнях иерархии. Декоратор разрешает добавлять новые обязанности по мере необходимости. Вместо того чтобы пытаться поддерживать все мыслимые возможности в одном сложном, допускающем разностороннюю настройку классе, вы можете определить простой класс и постепенно наращивать его функциональность с помощью декораторов. В результате приложение уже не платит за неиспользуемые функции. Нетрудно также определять новые виды декораторов независимо от классов, которые они расширяют, даже если первоначально такие расширения не планировались. При расширении же сложного класса обычно приходится вникать в детали, не имеющие отношения к добавляемой функции;

- позволяет избежать перегруженных функциями классов на верхних уровнях иерархии.

Декоратор разрешает добавлять новые обязанности по мере необходимости. Вместо того чтобы пытаться поддерживать все мыслимые возможности в одном сложном, допускающем разностороннюю настройку классе, вы можете определить простой класс и постепенно наращивать его функциональность с помощью декораторов. В результате приложение уже не платит за неиспользуемые функции. Нетрудно также определять новые виды декораторов независимо от классов, которые они расширяют, даже если первоначально такие расширения не планировались. При расширении же сложного класса обычно приходится вникать в детали, не имеющие отношения к добавляемой функции;

Паттерн Декоратор (Decorator). Результаты

- декоратор и его компонент не идентичны. Декоратор действует как прозрачное обрамление. Но декорированный компонент все же не идентичен исходному. При использовании декораторов это следует иметь в виду;
- множество мелких объектов. При использовании в проекте паттерна декоратор нередко получается система, составленная из большого числа мелких объектов, которые похожи друг на друга и различаются только способом взаимосвязи, а не классом и не значениями своих внутренних переменных. Хотя проектировщик, разбирающийся в устройстве такой системы, может легко настроить ее, но изучать и отлаживать ее очень тяжело.

Паттерн Декоратор (Decorator). Результаты

- **декоратор и его компонент не идентичны.** Декоратор действует как прозрачное обрамление. Но декорированный компонент все же не идентичен исходному. При использовании декораторов это следует иметь в виду;
- множество мелких объектов. При использовании в проекте паттерна декоратор нередко получается система, составленная из большого числа мелких объектов, которые похожи друг на друга и различаются только способом взаимосвязи, а не классом и не значениями своих внутренних переменных. Хотя проектировщик, разбирающийся в устройстве такой системы, может легко настроить ее, но изучать и отлаживать ее очень тяжело.

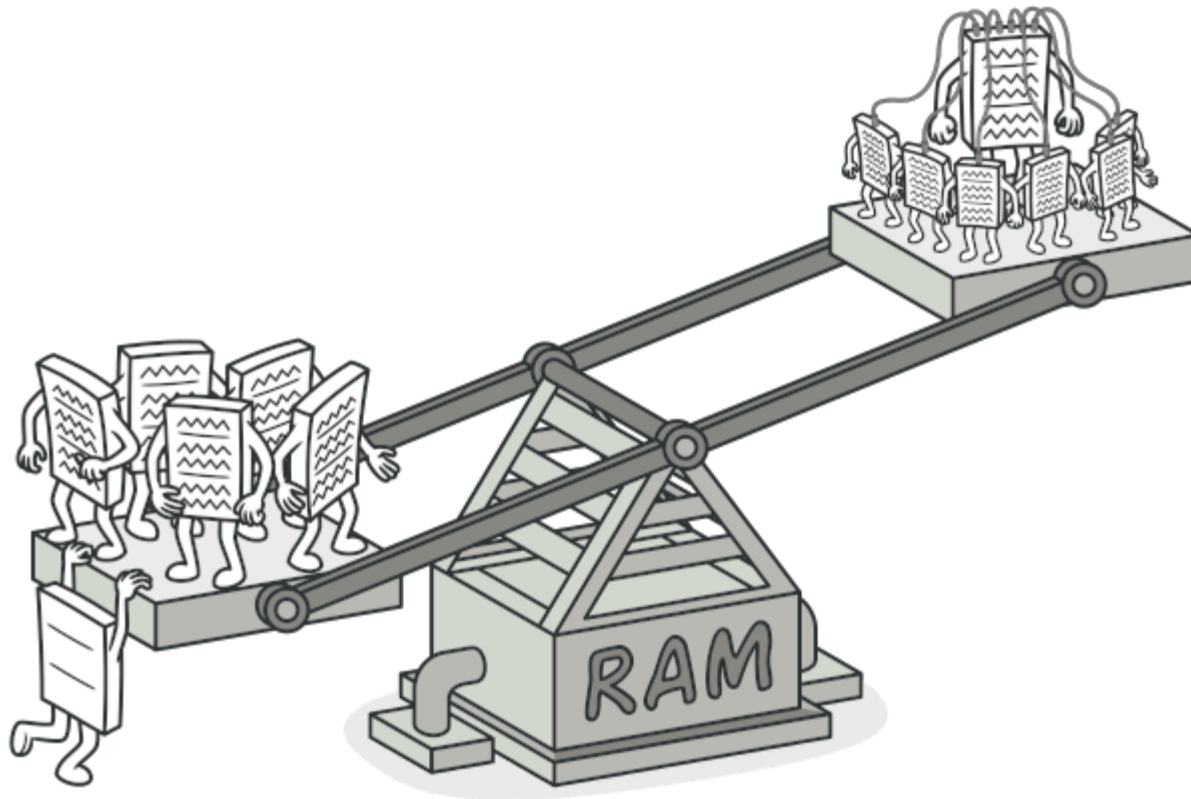
Паттерн Декоратор (Decorator). Результаты

- **декоратор и его компонент не идентичны.** Декоратор действует как прозрачное обрамление. Но декорированный компонент все же не идентичен исходному. При использовании декораторов это следует иметь в виду;
- **множество мелких объектов.** При использовании в проекте паттерна декоратор нередко получается система, составленная из большого числа мелких объектов, которые похожи друг на друга и различаются только способом взаимосвязи, а не классом и не значениями своих внутренних переменных. Хотя проектировщик, разбирающийся в устройстве такой системы, может легко настроить ее, но изучать и отлаживать ее очень тяжело.



Университет
Сириус
Колледж

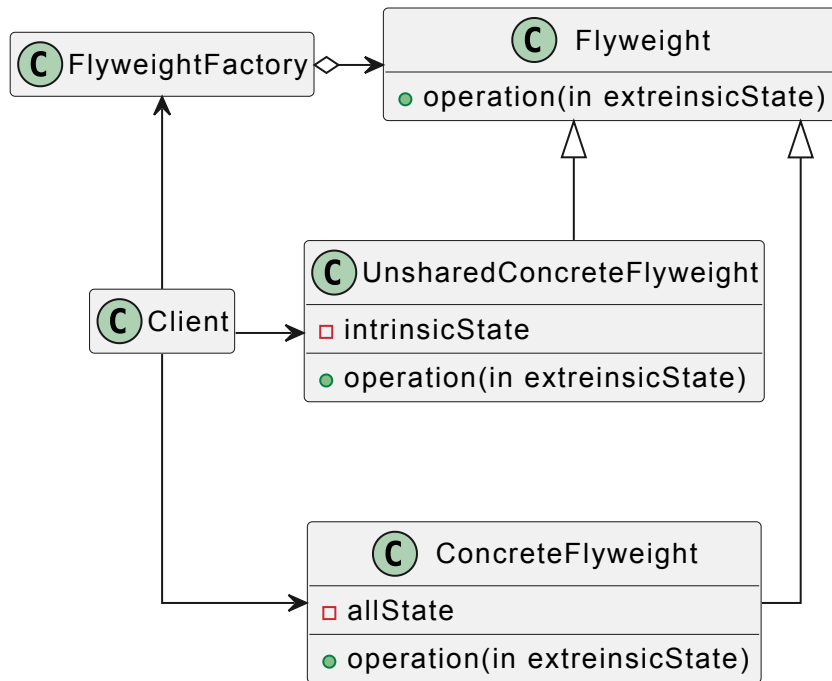
Паттерн Приспособленец (Flyweight)



Приспособленец - паттерн, структурирующий объекты. Использует разделение для эффективной поддержки множества мелких объектов.

Применяйте этот паттерн, когда выполнены все нижеперечисленные условия одновременно:

- в приложении используется большое число объектов;
- из-за этого накладные расходы на хранение высоки;
- большую часть состояния объектов можно вынести вовне;
- многие группы объектов можно заменить относительно небольшим количеством разделяемых объектов, поскольку внешнее состояние вынесено;
- приложение не зависит от идентичности объекта. Поскольку объекты приспособленцы могут разделяться, то проверка на идентичность возвратит «истину» для концептуально различных объектов.



```
1  import sys
2
3  # 1. Приспособленец (Flyweight)
4  # Хранит общее состояние (Intrinsic State).
5  # Эти объекты НЕИЗМЕНЯЕМЫ и разделяются между тысячами транзакций.
6  class MerchantType:
7      def __init__(self, name, category, icon_url, color_hex):
8          self.name = name
9          self.category = category
10         self.icon = icon_url
11         self.color = color_hex
12
13     def render(self, amount, date):
14         print(f"Отрисовка: {self.name} [{self.category}] | Сумма: {amount} | ")
15
```



```
2
3 # 1. Приспособленец (Flyweight)
4 # Хранит общее состояние (Intrinsic State).
5 # Эти объекты НЕИЗМЕНЯЕМЫ и разделяются между тысячами транзакций.
6 class MerchantType:
7     def __init__(self, name, category, icon_url, color_hex):
8         self.name = name
9         self.category = category
10        self.icon = icon_url
11        self.color = color_hex
12
13    def render(self, amount, date):
14        print(f"Отрисовка: {self.name} [{self.category}] | Сумма: {amount} | ")
15
16 # 2. Фабрика Приспособленцев
```

```
2
3 # 1. Приспособленец (Flyweight)
4 # Хранит общее состояние (Intrinsic State).
5 # Эти объекты НЕИЗМЕНЯЕМЫ и разделяются между тысячами транзакций.
6 class MerchantType:
7     def __init__(self, name, category, icon_url, color_hex):
8         self.name = name
9         self.category = category
10        self.icon = icon_url
11        self.color = color_hex
12
13    def render(self, amount, date):
14        print(f"Отрисовка: {self.name} [{self.category}] | Сумма: {amount} | ")
15
16 # 2. Фабрика Приспособленцев
```

```
7     def __init__(self, name, category, icon_url, color_hex):
8         self.name = name
9         self.category = category
10        self.icon = icon_url
11        self.color = color_hex
12
13    def render(self, amount, date):
14        print(f"Отрисовка: {self.name} [{self.category}] | Сумма: {amount} | ")
15
16 # 2. Фабрика Приспособленцев
17 class MerchantFactory:
18     _merchants = {}
19
20     @classmethod
21     def get_merchant_type(cls, name, category, icon, color):
```

```
14         print(f"Отрисовка: {self.name} [{self.category}] | Сумма: {amount} | ")
15
16 # 2. Фабрика Приспособленцев
17 class MerchantFactory:
18     _merchants = {}
19
20     @classmethod
21     def get_merchant_type(cls, name, category, icon, color):
22         key = (name, category)
23         if key not in cls._merchants:
24             cls._merchants[key] = MerchantType(name, category, icon, color)
25             print(f"[Factory] Создан новый тип магазина: {name}")
26         return cls._merchants[key]
27
28 # 3. Контекст (Context)
```

```
25         print(f"[Factory] Создан новый тип магазина: {name}")
26     return cls._merchants[key]
27
28 # 3. Контекст (Context)
29 class Transaction:
30     def __init__(self, amount, date, merchant_type: MerchantType):
31         self.amount = amount
32         self.date = date
33         self.merchant = merchant_type
34
35     def show_in_app(self):
36         self.merchant.render(self.amount, self.date)
37
38 # --- Использование ---
39
```

```
26         return cls._merchants[key]
27
28 # 3. Контекст (Context)
29 class Transaction:
30     def __init__(self, amount, date, merchant_type: MerchantType):
31         self.amount = amount
32         self.date = date
33         self.merchant = merchant_type
34
35     def show_in_app(self):
36         self.merchant.render(self.amount, self.date)
37
38 # --- Использование ---
39
40 # Загружаем историю операций (имитация БД)
```

```
29 class Transaction:
30     def __init__(self, amount, date, merchant_type: MerchantType):
31         self.amount = amount
32         self.date = date
33         self.merchant = merchant_type
34
35     def show_in_app(self):
36         self.merchant.render(self.amount, self.date)
37
38 # --- Использование ---
39
40 # Загружаем историю операций (имитация БД)
41 transactions_data = [
42     (150, "10:00", "Uber", "Taxi"),
43     (200, "10:30", "Uber", "Taxi"),
```

```
36         self.merchant.render(self.amount, self.date)
37
38 # --- Использование ---
39
40 # Загружаем историю операций (имитация БД)
41 transactions_data = [
42     (150, "10:00", "Uber", "Taxi"),
43     (200, "10:30", "Uber", "Taxi"),
44     (5000, "12:00", "Пятерочка", "Food"),
45     (300, "18:00", "Uber", "Taxi"),
46 ]
47
48 history = []
49
50 for amount, time, name, cat in transactions_data:
```



```
44         (5000, 12:00, пятерочка, good),
45         (300, "18:00", "Uber", "Taxi"),
46     ]
47
48     history = []
49
50     for amount, time, name, cat in transactions_data:
51         # Получаем легкий "общий" объект
52         m_type = MerchantFactory.get_merchant_type(name, cat, "icon.png", "#fff")
53         # Создаем транзакцию
54         t = Transaction(amount, time, m_type)
55         history.append(t)
56
57     print(f"\nВсего транзакций: {len(history)}")
58     print(f"Всего объектов MerchantType в памяти: {len(MerchantFactory._merchants")
```

```
44         (5000, 12:00, пятерочка, good),
45         (300, "18:00", "Uber", "Taxi"),
46     ]
47
48     history = []
49
50     for amount, time, name, cat in transactions_data:
51         # Получаем легкий "общий" объект
52         m_type = MerchantFactory.get_merchant_type(name, cat, "icon.png", "#fff")
53         # Создаем транзакцию
54         t = Transaction(amount, time, m_type)
55         history.append(t)
56
57     print(f"\nВсего транзакций: {len(history)}")
58     print(f"Всего объектов MerchantType в памяти: {len(MerchantFactory._merchants")
```

```
44         (5000, 12:00, пятерочка, good),
45         (300, "18:00", "Uber", "Taxi"),
46     ]
47
48     history = []
49
50     for amount, time, name, cat in transactions_data:
51         # Получаем легкий "общий" объект
52         m_type = MerchantFactory.get_merchant_type(name, cat, "icon.png", "#fff")
53         # Создаем транзакцию
54         t = Transaction(amount, time, m_type)
55         history.append(t)
56
57     print(f"\nВсего транзакций: {len(history)}")
58     print(f"Всего объектов MerchantType в памяти: {len(MerchantFactory._merchants")
```

При использовании приспособленцев не исключены затраты на передачу, поиск или вычисление внутреннего состояния, особенно если раньше оно хранилось как внутреннее. Однако такие расходы с лихвой компенсируются экономией памяти за счет разделения объектов-приспособленцев. Экономия памяти возникает по ряду причин:

- уменьшение общего числа экземпляров;
- сокращение объема памяти, необходимого для хранения внутреннего состояния;
- вычисление, а не хранение внешнего состояния (если это действительно так). Чем выше степень разделения приспособленцев, тем существеннее экономия.

С увеличением объема разделяемого состояния экономия также возрастает. Самого большого эффекта удастся добиться, когда суммарный объем внутренней и внешней информации о состоянии велик, а внешнее состояние вычисляется, а не хранится. Тогда разделение уменьшает стоимость хранения внутреннего состояния, а за счет вычислений сокращается память, отводимая под внешнее состояние.